



REST - REPRESENTATIONAL STATE TRANSFER



REPRESENTATIONAL STATE TRANSFER

- **REST** è uno stile architetturale per sistemi ipermediali distribuiti.
- È stato proposto da **Roy Fielding** nel 2000.
- Il suo obiettivo è **trasferire la rappresentazione delle risorse** da un componente (es. il server) a un altro (es. il client).



RESOURCE

- Una **risorsa** è qualsiasi informazione che possa essere nominata: un documento, un'immagine, un servizio, un oggetto non virtuale (es. una persona) o una collezione di altre risorse.
- Una risorsa è un insieme di elementi o valori che possono variare nel tempo
- Due risorse possono mappare agli stessi valori in un dato momento (Esempio: "versione v2.1" di un programma e "ultima versione" dello stesso programma).



REPRESENTATION

- La **rappresentazione della risorsa** è lo stato attuale o previsto di una risorsa, ovvero il valore della risorsa in un momento particolare.
- I componenti REST (client o server) eseguono azioni su una risorsa **utilizzando una rappresentazione**.
- La rappresentazione è composta da **dati e metadata**. Il formato dei dati è noto come "**media type**".



RESOURCE IDENTIFIERS

- Gli identificatori sono usati per **identificare**, ovvero indirizzare, una risorsa.
- Una **Uniform Resource Identifier (URI)** è una sequenza unica di caratteri che identifica una risorsa logica o fisica (Esempio: `http://example.com/users`).



URI BEST PRACTICES

Le migliori pratiche per i nomi delle URI:

1. Utilizzare **sostantivi** per rappresentare le risorse.
2. Utilizzare **sostantivi singolari** per una singola risorsa (Esempio:
`http://example.com/users/admin`).
3. Utilizzare **sostantivi plurali** per una collezione di risorse (Esempio:
`http://example.com/users/`).



URI NOTATION

Convenzioni di notazione per le URI:

1. **Forward slash (/)**: Usato per esprimere la gerarchia.
 1. Suggerimento: usare il *trailing slash* solo se la risorsa non è una foglia (leaf).
2. **Preferire i trattini (-)** agli underscore (_).
3. Utilizzare **solo lettere minuscole**.
4. Non utilizzare **estensioni di file** (il media type è comunicato negli header).
5. Utilizzare la **componente query** per filtrare (Esempio:
`http://example.com/managed-devices/?region=USA`).



PRINCIPI DI FORMULAZIONE URI

Regola	Esempio Positivo	Esempio Negativo
Risorsa Singola (Singolare)	/users/45	/get-user/45 (Contiene un verbo)
Collezione (Plurale)	/invoices	/invoice-list
Gerarchia	/users/45/orders	/orders-from-user/45
Separazione	Preferire i trattini (-)	Evitare gli underscore (_)
☐ Media Type	Non usare estensioni (es. .json)	/products/123.json

OPERATIONS: NOT IN THE URI

- Le URI sono usate per identificare in modo univoco le **risorse** e **non le azioni** su di esse.
- Azioni diverse possono essere eseguite su una risorsa attraverso i **metodi supportati** (l'interfaccia tra i componenti).
- Esempi di metodi sulla stessa URI: **GET**, **PUT**, e **DELETE** (es. `http://example.com/managed-devices/{id}`).



URI

protocol://subdomain.domain.tld/path?query

<https://www.google.com/maps/?hl=it>



IN PYTHON

```
import requests

base_url = "https://api.example.com/api/v1/products"
query_params = {
    'country': 'Italy', 'min_price': 50.00,
    'sort_by': 'price_asc'
}

# https://api.example.com/api/v1/products?
# country=Italy&min_price=50.00&sort_by=price_asc
response = requests.Request('GET', base_url, params=query_params).prepare()
print(f"URI generato con Query: {response.url}")
```



RESTFUL SYSTEM CONSTRAINTS

I **vincoli** di un sistema RESTful sono:

1. **client-server**
2. **stateless**
3. **cacheable**
4. **uniform-interface**
5. **layered system**



CLIENT-SERVER

- Applica la **separazione delle responsabilità** (il client gestisce l'UI, il server gestisce l'archiviazione dei dati).
- Migliora la **portabilità** e la **scalabilità**.



STATELESS

- Ogni richiesta del client deve contenere **tutte le informazioni** necessarie per essere compresa.
- Non può sfruttare alcun **contesto memorizzato** sul server.
- Lo stato della sessione è mantenuto interamente sul **client** (lo stato della risorsa è mantenuto sul server).



CACHEABLE

- Il client può in seguito **riutilizzare una rappresentazione** della risorsa (dati) che è considerata *cacheable*.
- Il periodo di tempo per cui la risorsa può essere messa in cache è specificato nella risposta.



UNIFORM INTERFACE

- Un'interfaccia uniforme tra i componenti promuove la **standardizzazione** (a discapito dell'efficienza).
- Le implementazioni sono disaccoppiate dai servizi che forniscono.
- I quattro vincoli dell'interfaccia sono:
 - Identificazione delle risorse.
 - Manipolazione delle risorse tramite rappresentazioni.
 - Messaggi auto-descrittivi.
 - Hypermedia come motore dello stato dell'applicazione (il client necessita solo della URI iniziale).



UNIFORM INTERFACE (ES)

- API REST basate su HTTP usano metodi standard (GET, POST, PUT, DELETE, etc.) e gli URI per identificare risorse.



LAYERED SYSTEM

- Nella comunicazione possono essere coinvolti diversi componenti in un'architettura a strati (ad esempio: *origin server, gateway, proxy, user agent*).
- I componenti intermedi agiscono sia come client che come server; inoltrano richieste e risposte, a volte con una traduzione.
- Ogni componente **non può "vedere" oltre lo strato immediatamente adiacente** con cui interagisce.



HTTP VS. REST



I METODI HTTP (VERBI REST)

In un'architettura RESTful, i Metodi HTTP (noti come "Verbi") definiscono l'azione che si desidera eseguire sulla Risorsa (URI).

Gli URI identificano la **cosa** (sostantivo), i Metodi HTTP definiscono **cosa fare** con quella cosa.



Esempio:

Metodo	Funzione	Corrispondenza CRUD
GET	Recupera una risorsa o una collezione.	Read
POST	Crea una nuova risorsa in una collezione.	Create
PUT	Sostituisce completamente una risorsa esistente.	Update/Replace
DELETE	Rimuove una risorsa specifica.	Delete
PATCH	Applica modifiche parziali a una risorsa.	Update/Modify



CODICI DI SUCCESSO RESTFUL COMUNI

- 200 **OK**: Utilizzato per richieste GET, PUT, PATCH o DELETE eseguite con successo.
- 201 **Created**: **MUST** essere restituito dopo un **POST** di successo che crea una nuova risorsa. La risposta dovrebbe includere la nuova risorsa creata.
- 204 **No Content**: Restituito quando l'azione ha avuto successo, ma non c'è corpo di risposta da inviare al client (tipico per DELETE o PUT senza contenuto di ritorno).



CODICI DI ERRORE CLIENTE COMUNI (4XX)

- **400 Bad Request:** La richiesta non può essere soddisfatta a causa di un errore del client (es. JSON malformato, dati mancanti).
- **401 Unauthorized:** Richiesta rifiutata perché manca l'autenticazione.
- **403 Forbidden:** L'utente è autenticato, ma non ha i permessi per accedere a quella risorsa.
- **404 Not Found:** La risorsa richiesta non esiste.



AZIONE O RISORSA?



AZIONE I: OTTENERE DATI

- **Obiettivo:** Ottenere tutti i prodotti in vendita.
- **Design corretto:** GET /products
- **Motivazione:** Si sta recuperando la collezione di prodotti.



AZIONE 2: CREARE UNA NUOVA RISORSA

- **Obiettivo:** Aggiungere un nuovo utente.
- **Design corretto:** POST /users (corpo della richiesta contiene i dati del nuovo utente)
- **Risposta attesa:** 201 Created. L'API dovrebbe restituire la nuova risorsa e l'URI per accedervi.



AZIONE 3: ESEGUIRE UNA FUNZIONE SPECIFICA

Se un'azione non è mappabile a CRUD (es. "Pubblica un documento" o "Cambia password"), ci sono due approcci accettati:

1. **Modeling come Risorsa:** Modellare l'azione come una risorsa.

- **Esempio:** Pubblicare un documento.
- **URI:** POST /documents/{id}/publish

2. **PATCH (Modifica Stato):** Utilizzare PATCH per aggiornare un attributo di stato.

- **Esempio:** Pubblicare un documento aggiornando lo stato.
- **URI:** PATCH /documents/{id} (Corpo: {"status": "published"})

□ Secondo approccio è preferito perché mantiene un modello più puro, in cui si agisce sullo **stato** della risorsa.

REFERENCES

- https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- <https://restfulapi.net>
- <https://datatracker.ietf.org/doc/html/rfc3986>
- https://en.wikipedia.org/wiki/Uniform_Resource_Identifier



